Metodologías de desarrollo y levantamiento de requisitos de software

Breve descripción:

Este componente formativo brinda las bases conceptuales y prácticas para comprender la gestión del desarrollo de software y el levantamiento de requisitos. Aborda metodologías tradicionales y ágiles, tipos de requisitos, técnicas de elicitación y roles del proceso, fortaleciendo el criterio técnico y comunicativo en equipos de desarrollo.

Octubre 2026

Tabla de contenido

Introducción 4

1. Metodologías de desarrollo de software 6

1.1. Metodologías ágiles y tradicionales 8

1.2. Características, ventajas y desventajas de las metodologías 13

2. Fundamentos de los requisitos de software 19

2.1. Concepto de requisitos de software 21

2.2. Tipos y características de los requisitos 25

3. Elicitación de requisitos de software 32

3.1. Concepto y análisis de documentos 33

4. Técnicas para la recolección de información 36

4.1. Técnicas de recolección de información basadas en interacción y observación 37

4.2. Focus group como técnica de recopilación de información 42

4.3. Talleres de Requisitos (JAD - Joint Application Development) 46

5. Roles en el desarrollo y levantamiento de requisitos 55

5.1. Usuarios, actores y stakeholders 55

5.2. Cliente líder, dueño del producto, equipo de desarrollo y analista 59

Síntesis 67

Glosario 69

Referencias bibliográficas 71

Créditos 72

Introducción

El desarrollo de software es una de las actividades más complejas y estratégicas del mundo contemporáneo. A medida que las organizaciones dependen cada vez más de los sistemas de información para operar, competir y crecer, la capacidad de construir software de calidad, en el tiempo y presupuesto acordados, se convierte en una ventaja competitiva determinante. Sin embargo, la historia de la ingeniería de software está plagada de proyectos fallidos, retrasados o que no satisfacen las necesidades reales de sus usuarios.

Ante este panorama, dos disciplinas se erigen como pilares fundamentales del éxito en los proyectos de software:

- **Las metodologías**: proporcionan el marco de trabajo que organiza, guía y controla el proceso de construcción del software.
- **El levantamiento de requisitos**: asegura que el sistema que se construye responde efectivamente a las necesidades del cliente y los usuarios finales.

Por lo anterior, este componente formativo está diseñado para que se adquieran las competencias necesarias para comprender, seleccionar y aplicar metodologías de desarrollo de software, así como para dominar las técnicas de levantamiento y gestión de requisitos. El componente aborda desde los conceptos fundamentales hasta las técnicas prácticas de recolección de información, pasando por los roles que intervienen en estos procesos.

El dominio de estos conocimientos y habilidades es esencial para el desempeño profesional en contextos de desarrollo de software, ya que permite actuar como puente entre las necesidades del negocio y las soluciones tecnológicas, traduciendo los

requerimientos de los usuarios en especificaciones técnicas claras que orienten al equipo de trabajo hacia la construcción de productos de software de calidad.

1. Metodologías de desarrollo de software

Una metodología de desarrollo de software es un conjunto estructurado de prácticas, principios, técnicas y herramientas que orientan al equipo de desarrollo en la producción de software de calidad, dentro de los plazos y presupuestos establecidos. Las metodologías no son fórmulas rígidas, sino marcos de referencia que deben adaptarse al contexto específico de cada proyecto y organización.

La importancia de contar con una metodología radica en que proporciona un lenguaje común al equipo, define responsabilidades claras, establece puntos de control para medir el avance y permite gestionar los riesgos de manera proactiva. Sin una metodología, los proyectos de software tienden a caer en el desarrollo ad hoc, donde la falta de estructura genera inconsistencias, retrabajo y productos de baja calidad que no satisfacen al cliente.

Históricamente, el desarrollo de software ha transitado por varias etapas:

- **Años 60–70. Desarrollo artesanal**

El desarrollo de software se realizaba de forma empírica, sin metodologías definidas ni procesos estandarizados. El código se construía de manera manual, dependiente de la experiencia individual del programador.

- **Años 70–80. Modelos secuenciales**

Surgen metodologías formales como el modelo en cascada y el modelo espiral, orientadas a aplicar principios de ingeniería, control y documentación al proceso de desarrollo de software.

- **Años 90. Enfoques flexibles**

Aparecen modelos que buscan mayor adaptabilidad frente al cambio, cuestionando la rigidez de los enfoques tradicionales y priorizando la iteración y la retroalimentación continua.

- **2001. Manifiesto Ágil**

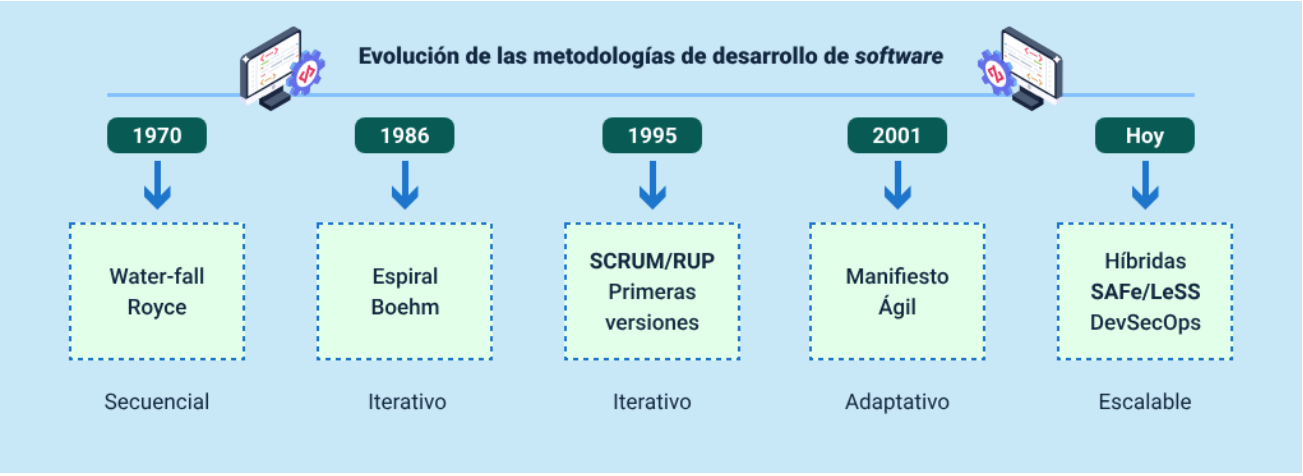
La publicación del Manifiesto Ágil marca un punto de inflexión, consolidando principios centrados en la colaboración, la entrega incremental y la respuesta al cambio.

- **Actualidad. Enfoques híbridos y escalado ágil**

Predominan enfoques que combinan metodologías tradicionales y ágiles, junto con frameworks de escalado ágil utilizados en proyectos empresariales de gran tamaño.

Para complementar lo anterior, se presenta la siguiente imagen que de manera exacta indica la evolución metodológica por años:

Figura 1. Línea de tiempo de la evolución de las metodologías de desarrollo de software



1.1. Metodologías ágiles y tradicionales

Las metodologías de desarrollo se clasifican en dos grandes familias: las tradicionales (predictivas o pesadas) y las ágiles (adaptativas o livianas). Esta clasificación no es dicotómica ni absoluta; existen metodologías híbridas que combinan elementos de ambos enfoques para adaptarse a contextos específicos de las organizaciones y los proyectos.

Entender las diferencias fundamentales entre estos dos enfoques no es un ejercicio meramente académico: es una competencia práctica que el analista y el equipo de desarrollo deben dominar para tomar decisiones informadas al inicio de cada proyecto. La elección de una metodología inadecuada puede comprometer el éxito del proyecto desde sus primeras semanas, generando fricciones con el cliente, desmotivación en el equipo y productos que no responden a las necesidades reales del negocio.

En la práctica profesional actual, la dicotomía entre ágil y tradicional ha dado paso a un espectro más amplio y matizado. Organizaciones como el Ejército de los Estados Unidos, que históricamente era el bastión del desarrollo tradicional, han adoptado principios ágiles en sus proyectos de software. A su vez, equipos que trabajan con SCRUM incorporan elementos de disciplina documental propios de los enfoques tradicionales cuando el contexto regulatorio o contractual lo exige.

Conocer en profundidad ambas familias metodológicas le permite al profesional navegar este espectro con criterio, seleccionando y combinando los elementos que mejor respondan a las características únicas de cada proyecto y organización.

A. Metodologías tradicionales

Nacieron en una época en la que el software se concebía y construía de manera similar a la infraestructura física: a partir de planos detallados, fases secuenciales bien definidas y contratos de alcance fijo. Este enfoque resultó eficaz mientras los requisitos eran estables, los entornos poco cambiantes y los sistemas relativamente simples. Sin embargo, a medida que el software se convirtió en un factor clave de la innovación empresarial y los mercados comenzaron a transformarse con mayor rapidez, la rigidez de estos modelos empezó a evidenciar limitaciones significativas, las cuales serían cuestionadas de forma directa por el Manifiesto Ágil en 2001.

Las metodologías tradicionales surgieron antes de la década de 1990 y se caracterizan por una planificación exhaustiva al inicio del proyecto, fases secuenciales bien delimitadas y documentación extensa. Estas metodologías destacan los siguientes modelos o enfoques:

- **Modelo en cascada (waterfall)**

Propuesto por Winston Royce en 1970, es el representante paradigmático de este enfoque. Sus fases son secuenciales y dependientes: análisis de requisitos, diseño del sistema, implementación, pruebas, despliegue y mantenimiento. La premisa central es que los requisitos son conocidos, estables y completamente definibles desde el inicio, lo que permite planificar con precisión el alcance, el costo y el tiempo total del proyecto.

- **Modelo en espiral**

Creado por Barry Boehm en 1986, introdujo un avance significativo al incorporar la gestión explícita de riesgos en ciclos iterativos. Cada espiral se compone de cuatro

cuadrantes: determinación de objetivos y alternativas, análisis y resolución de riesgos, desarrollo y prueba, y planificación del siguiente ciclo. Este modelo es especialmente valioso en proyectos de gran escala con alta incertidumbre técnica, donde identificar y mitigar riesgos en cada iteración es tan crítico como cumplir con el plan de desarrollo.

- **Rational Unified Process (RUP) - Proceso Unificado de Rational**

Desarrollado por Booch, Jacobson y Rumbaugh en 1999, organiza el desarrollo en cuatro fases (Inicio, Elaboración, Construcción, Transición) con múltiples iteraciones dentro de cada una. RUP hace uso extensivo de UML para el modelado y produce una documentación detallada en cada fase. Es considerado un enfoque híbrido que incorpora iteraciones, pero mantiene una carga documental significativa. Las metodologías tradicionales son especialmente apropiadas para proyectos con requisitos muy estables, como sistemas de control industrial o proyectos del sector aeroespacial, de defensa o de salud con regulaciones estrictas, donde la predictibilidad supera con creces las limitaciones de flexibilidad.

B. Metodologías ágiles

No surgieron como una tendencia circunstancial, sino como una respuesta empírica y pragmática a los reiterados fracasos de proyectos de software, ampliamente documentados durante décadas por los informes CHAOS del Standish Group. Su premisa central sostiene que, en contextos de alta incertidumbre y cambio constante, la capacidad de adaptación, la colaboración continua y la entrega incremental de valor resultan más efectivas que la adhesión estricta a planes definidos con información incompleta desde el inicio.

Las metodologías ágiles emergieron como respuesta directa a las limitaciones del enfoque tradicional en proyectos con requisitos volátiles o poco definidos. El Manifiesto Ágil (2001), firmado por diecisiete expertos reunidos en Snowbird, Utah, estableció cuatro valores: individuos e interacciones sobre procesos y herramientas; software funcionando sobre documentación exhaustiva; colaboración con el cliente sobre negociación contractual; y respuesta ante el cambio sobre seguir un plan. Estos valores se complementan con doce principios que orientan la práctica cotidiana del desarrollo ágil.

SCRUM es el framework ágil más adoptado globalmente. Organiza el trabajo en sprints de 1 a 4 semanas, con tres roles:

- **Product Owner (PO)**: responsable de la visión del producto y la priorización del backlog.
- **Scrum master**: facilitador y guardián de los valores ágiles.
- **Development Team**: equipo multifuncional y autoorganizado.

Además de lo anterior, este marco de trabajo incorpora cuatro ceremonias — Sprint Planning, Daily Scrum, Sprint Review y Sprint Retrospective— que garantizan la transparencia del proceso, la inspección continua y la adaptación permanente al cambio. De manera complementaria, Extreme Programming (XP) pone un énfasis particular en las buenas prácticas técnicas, tales como el desarrollo guiado por pruebas (TDD), la programación en pareja y la integración continua, con el fin de mejorar la calidad del software. Por su parte, Kanban, adaptado del sistema de producción Toyota, se enfoca en la gestión del flujo de trabajo mediante tableros visuales y límites al trabajo en progreso (WIP), lo que permite identificar y eliminar cuellos de botella. En conjunto, estas metodologías ágiles no son excluyentes, sino que pueden combinarse

estratégicamente para aprovechar sus complementariedades según las necesidades del proyecto.

La siguiente tabla presenta una comparación sistemática de las dimensiones más relevantes entre los dos grandes enfoques metodológicos. Esta comparación permite al analista identificar rápidamente cuál enfoque se alinea mejor con las características de un proyecto determinado:

Tabla 1. Comparación entre metodologías tradicionales y ágiles

Dimensión	Metodologías tradicionales	Metodologías ágiles
Planificación.	Exhaustiva al inicio.	Iterativa y adaptativa.
Documentación.	Extensa y formal.	Mínima e indispensable.
Requisitos.	Fijos desde el inicio.	Variables y negociables.
Entregas.	Una entrega al final.	Entregas frecuentes (sprints).
Equipo.	Roles muy especializados.	Equipos multifuncionales.
Cliente.	Involucrado al inicio y al final.	Involucrado continuamente.
Cambios.	Costosos y difíciles.	Bienvenidos y esperados.
Riesgo.	Detectado tardíamente.	Gestionado en cada iteración.
Idóneo para.	Requisitos estables, alta regulación.	Requisitos cambiantes, innovación.

1.2. Características, ventajas y desventajas de las metodologías

La selección de una metodología adecuada es una de las decisiones más importantes al inicio de un proyecto de software. No existe una metodología universalmente superior; la elección óptima depende de múltiples factores contextuales que el analista debe evaluar cuidadosamente. Una decisión equivocada puede condicionar negativamente todo el proyecto desde sus fases iniciales.

Los factores más relevantes para la selección incluyen: el tamaño y complejidad del proyecto, la estabilidad y claridad de los requisitos, la distribución geográfica del equipo, la cultura organizacional, las regulaciones del sector, el nivel de experiencia del equipo con metodologías ágiles o tradicionales, el tipo de contrato (precio fijo o tiempo y materiales) y las expectativas del cliente en cuanto a visibilidad y participación en el proceso de desarrollo.

- **Ventajas y desventajas del enfoque tradicional**

Las metodologías tradicionales ofrecen ventajas significativas en contextos específicos:

a) Alta predictibilidad del proyecto

La predictibilidad constituye la mayor fortaleza de las metodologías tradicionales, ya que al definir completamente el alcance antes de iniciar el desarrollo es posible estimar con mayor precisión los costos y los tiempos. Esto facilita la gestión de contratos de precio fijo y la planificación presupuestal a largo plazo.

b) Documentación exhaustiva

La producción de documentación detallada facilita el mantenimiento del sistema a lo largo del tiempo, especialmente cuando el equipo de desarrollo original ya no se encuentra disponible.

c) Claridad en roles y responsabilidades

La separación del proceso en fases bien definidas reduce la ambigüedad sobre las responsabilidades de cada actor, estableciendo con claridad quién hace qué en cada etapa.

d) Puntos formales de control y seguimiento

La aprobación formal entre fases proporciona hitos de control que permiten a los patrocinadores y responsables del proyecto tener mayor visibilidad sobre el avance y la toma de decisiones.

Sus principales desventajas son igualmente significativas:

a) Rigidez frente al cambio

La incorporación de modificaciones en fases avanzadas del proyecto resulta especialmente costosa, pudiendo implicar un esfuerzo entre 10 y 100 veces mayor que si los cambios se hubieran identificado durante la fase de análisis.

b) Entrega tardía de software funcional

El producto operativo se entrega únicamente al final del proyecto, lo que genera largos periodos sin retroalimentación real por parte del usuario y aumenta el riesgo de que el resultado final no satisfaga las necesidades reales.

c) **Detección tardía de problemas de integración**

Los errores y conflictos entre componentes suelen identificarse en etapas finales, cuando su corrección implica mayores costos y complejidad técnica.

d) **Alta carga documental y burocrática**

La extensa documentación requerida puede ralentizar el proceso de desarrollo y afectar la motivación del equipo técnico, especialmente en proyectos de mediana escala donde dicha carga no aporta un valor proporcional.

Ejemplo práctico:

Una empresa colombiana que contrata el desarrollo de un sistema de nómina para el gobierno departamental bajo contrato de precio fijo con entrega en 18 meses, es un escenario típico donde el enfoque tradicional es apropiado. Los requisitos legales de liquidación de nómina son estables y bien conocidos, el cliente no puede participar semanalmente en el proceso, y la obligación contractual de entregar documentación completa hace de Waterfall o RUP la elección natural.

- **Ventajas y desventajas del enfoque ágil**

Las metodologías ágiles destacan por lo siguiente:

a) **Alta adaptabilidad al cambio**

La capacidad de adaptarse rápidamente al cambio constituye la ventaja más diferenciadora de las metodologías ágiles, especialmente en entornos donde los negocios evolucionan con rapidez y los usuarios descubren sus necesidades reales al interactuar con prototipos y versiones tempranas del sistema.

b) Entrega frecuente de valor

La entrega continua de incrementos funcionales en ciclos cortos permite obtener retroalimentación real del cliente en semanas en lugar de meses o años, reduciendo significativamente el riesgo de construir un producto que no responda a las necesidades del negocio.

c) Colaboración constante con el cliente

La interacción permanente con el cliente fomenta un mayor compromiso y sentido de propiedad sobre el producto, facilitando la toma de decisiones oportunas y alineadas con los objetivos del proyecto.

d) Motivación y productividad del equipo

Los equipos ágiles suelen presentar mayores niveles de motivación y satisfacción laboral, lo que se traduce en menor rotación de personal y una productividad más sostenible a lo largo del tiempo.

Sus limitaciones son igualmente reales y deben gestionarse conscientemente:

- **Dificultad en la estimación del costo total**

Al definirse el alcance de manera progresiva, la estimación precisa del costo total del proyecto resulta compleja, lo que puede generar tensiones en contratos de precio fijo y dificultar la planificación presupuestal a largo plazo.

- **Menor formalización documental**

La reducción de documentación formal puede complicar el mantenimiento del sistema y la transferencia de conocimiento cuando se producen cambios en el equipo de trabajo.

- **Alta dependencia del compromiso del cliente**

Los enfoques ágiles requieren una participación continua y activa del cliente, lo cual no siempre es viable en organizaciones con alta carga operativa o disponibilidad limitada.

- **Desafíos de escalabilidad**

La aplicación de metodologías ágiles en proyectos de gran escala, con equipos numerosos y distribuidos geográficamente, demanda el uso de frameworks adicionales como SAFe o LeSS, los cuales implican una curva de aprendizaje y costos de implementación significativos.

Ejemplo práctico:

Una startup fintech colombiana que está construyendo una aplicación móvil de pagos digitales, con requisitos que evolucionan semanalmente según el feedback de sus primeros usuarios y un equipo de 8 personas co-ubicadas, es el escenario ideal para SCRUM. La capacidad de iterar rápidamente y ajustar la dirección del producto según el aprendizaje de cada sprint es el factor diferenciador que puede determinar el éxito en un mercado tan competitivo y cambiante.

Partiendo de lo previamente expuesto, la siguiente tabla amplía un análisis comparativo, incluyendo metodologías específicas con sus ventajas, limitaciones y

contextos de aplicación óptimos. Esta referencia es de gran utilidad al momento de fundamentar la selección metodológica ante el equipo y los patrocinadores del proyecto:

Tabla 2. Metodologías de desarrollo: análisis comparativo detallado

Metodología	Tipo	Iteraciones	Ventaja clave	Limitación principal	Mejor contexto
Waterfall.	Tradicional.	No.	Predictibilidad.	Rigidez al cambio.	Requisitos fijos, regulado.
Espiral.	Tradicional.	Sí (fases).	Gestión de riesgos.	Alta complejidad.	Proyectos de alto riesgo.
RUP.	Híbrido.	Sí.	Flexibilidad iterativa.	Documentación extensa.	Empresas grandes.
SCRUM.	Ágil.	Sprints 1-4 semanas.	Adaptabilidad rápida.	Difícil escalar.	Productos innovadores.
XP.	Ágil.	Semanas.	Calidad técnica.	Requiere alta disciplina.	Énfasis en código.
Kanban.	Ágil.	Flujo continuo.	Visualización flujo.	Sin cadencia definida.	Soporte/mantenimiento.

2. Fundamentos de los requisitos de software

Los requisitos de software constituyen la base sobre la cual se construye cualquier sistema de información. Son la expresión formal de las necesidades, expectativas y restricciones que el sistema debe satisfacer. El IEEE los define como: una condición o capacidad que debe satisfacer o poseer un sistema para cumplir con un contrato, estándar, especificación u otro documento impuesto formalmente.

Una gestión deficiente de los requisitos es la causa más frecuente de fracaso en proyectos de software. Según el Standish Group (2015), más del 60 % de los fallos están relacionados con problemas en la definición y gestión de requisitos. Barry Boehm demostró que el costo de corregir un error en los requisitos crece exponencialmente: un error detectado en análisis puede costar 1 unidad; el mismo error en producción puede costar entre 100 y 200 veces más. Esta evidencia subraya por qué invertir tiempo y recursos en una ingeniería de requisitos sólida es siempre rentable.

Detrás de estas cifras hay una realidad que cualquier profesional del software reconoce con facilidad:

Los proyectos no fracasan porque los programadores escriban mal código, sino porque el equipo construye el sistema incorrecto.

Se implementan funcionalidades que nadie usa, se omiten capacidades que el negocio considera indispensables, se asumen comportamientos del sistema que el cliente jamás aprobó, y se descubren inconsistencias críticas cuando el producto ya está en producción y el costo de corregirlas es devastador. Todos estos problemas tienen un origen común:

Una ingeniería de requisitos deficiente que no identificó, documentó y validó correctamente lo que el sistema debía hacer antes de comenzar a construirlo.

Los requisitos son, en esencia, el lenguaje mediante el cual el mundo del negocio y el mundo de la tecnología se comunican. Son el contrato implícito —y en muchos casos explícito— entre quien encarga el sistema y quien lo construye. Por ejemplo:

a) Bien redactado

Cuando el contrato está bien redactado, es claro, completo, consistente y verificable, el proceso de desarrollo avanza con mayor seguridad y confianza. Las decisiones de diseño e implementación se toman con base en criterios definidos, se reducen los malentendidos y se facilita la validación del producto frente a las expectativas del cliente.

b) Mal redactado

Cuando está mal redactado, es ambiguo o incompleto, cada decisión de diseño e implementación se convierte en una apuesta: el desarrollador asume lo que el cliente quería, el cliente asume que el desarrollador entendió su intención, y el resultado final satisface a ninguno de los dos.

La ingeniería de requisitos como disciplina formal surge precisamente para estructurar y sistematizar ese proceso de comunicación entre el negocio y la tecnología. No se trata de llenar formularios ni de generar documentación por obligación contractual: se trata de construir un entendimiento compartido, profundo y verificable de lo que el sistema debe lograr, bajo qué condiciones debe operar y con qué nivel de calidad debe hacerlo. Este entendimiento compartido es la materia prima sin la cual

ninguna metodología de desarrollo —ágil o tradicional— puede producir resultados satisfactorios de manera consistente.

Es importante también comprender que los requisitos no son estáticos. En proyectos de mediana y larga duración, los requisitos evolucionan a medida que el negocio cambia, los usuarios descubren nuevas necesidades al interactuar con prototipos, y el entorno regulatorio se transforma. Por ello, la gestión de requisitos no termina con la elaboración del documento de especificación inicial: es un proceso continuo de mantenimiento, control de cambios y trazabilidad que acompaña al proyecto durante todo su ciclo de vida.

Dominar tanto la identificación inicial de requisitos como su gestión evolutiva es una competencia que diferencia al analista de requisitos verdaderamente experto del que apenas conoce las técnicas básicas de elicitación.

2.1. Concepto de requisitos de software

Desde una perspectiva práctica, un requisito es cualquier característica observable o medible que el sistema debe poseer para ser aceptado por el cliente. Los requisitos representan un contrato entre el cliente y el equipo de desarrollo: definen qué se construirá, bajo qué condiciones operará y qué calidad tendrá el producto final. Su correcta definición es una precondición para que el resto del proceso de desarrollo tenga éxito.

Los requisitos pueden expresarse en lenguaje natural, mediante diagramas UML, modelos formales o una combinación de estos. La clave es que sean comprensibles tanto para los stakeholders del negocio como para el equipo técnico. Un requisito

bien expresado guía al diseñador, al programador y al tester, creando un hilo de trazabilidad que conecta la necesidad del negocio con el código del sistema y con las pruebas que verifican su correcto funcionamiento.

- **Propiedades de un buen requisito**

Un requisito de calidad debe cumplir un conjunto de propiedades bien establecidas en la literatura de la ingeniería de software. El estándar IEEE 830 (práctica recomendada para especificaciones de requisitos de software) y su sucesor ISO/IEC/IEEE 29148 definen estas propiedades de manera formal. Su importancia radica en que cada propiedad que falta en un requisito se convierte en una fuente potencial de malentendidos, retrabajo o conflictos entre el cliente y el equipo de desarrollo. A continuación, se describen las ocho propiedades fundamentales:

a) Correcto

El requisito refleja con precisión la necesidad real del stakeholder, sin distorsiones, suposiciones ni interpretaciones personales del analista. Un requisito incorrecto lleva a implementar funcionalidades que el usuario no necesita.

b) Completo

Incluye toda la información necesaria para su comprensión e implementación sin requerir conocimiento implícito o tácito. Un requisito incompleto obliga al desarrollador a tomar decisiones de negocio que no le corresponden.

c) Consistente

No contradice ni entra en conflicto con otros requisitos del sistema. Los conflictos entre requisitos son frecuentes cuando provienen de diferentes grupos de stakeholders y deben resolverse con el cliente antes de la implementación.

d) No ambiguo

Tiene una única interpretación posible para todos los involucrados en el proyecto, independientemente de su perfil técnico o de negocio. Las palabras como 'rápido', 'fácil', 'robusto' o 'amigable' son inherentemente ambiguas y deben cuantificarse.

e) Verificable

Puede ser probado mediante criterios de aceptación concretos y medibles para confirmar que el sistema lo cumple. Un requisito que no puede probarse no puede garantizarse.

f) Trazable

Puede vincularse claramente a su origen (stakeholder específico, documento de negocio o regulación) y a los artefactos de diseño, código y pruebas que lo implementan. La trazabilidad es fundamental para gestionar el impacto de los cambios.

g) Factible

Puede ser implementado dentro de las restricciones técnicas, económicas y temporales del proyecto. Requisitos técnicamente imposibles o económicamente inviables deben identificarse tempranamente y renegociarse con el cliente.

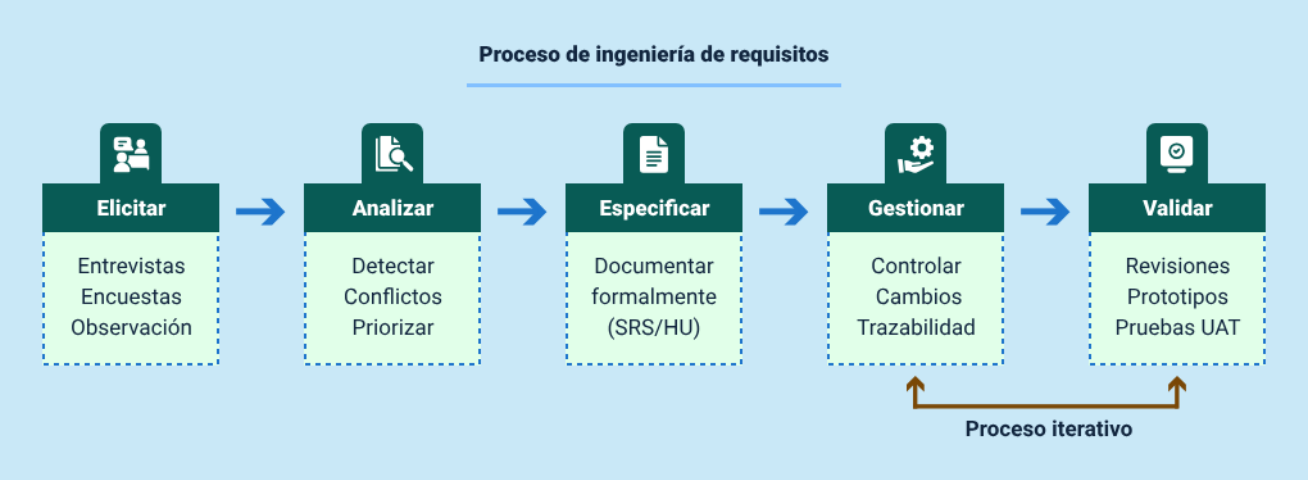
h) Priorizable

Puede ser ordenado según su importancia relativa para el negocio, permitiendo decisiones informadas de alcance cuando el presupuesto o el tiempo no alcanza para implementar todo lo deseado.

En la práctica, el analista debe revisar cada requisito documentado contra estas ocho propiedades antes de presentarlo al cliente para su aprobación. Una técnica útil es la revisión por pares (peer review) de los requisitos, donde un colega analista lee cada requisito y señala las propiedades que no se cumplen. Esta práctica, aunque sencilla, detecta la mayoría de los defectos de calidad en los requisitos antes de que se conviertan en problemas costosos durante el desarrollo o las pruebas.

Para ejemplificar el proceso de ingeniería de requisitos, se relaciona un esquema que ilustra las cinco fases del proceso de ingeniería de requisitos y sus interrelaciones. Es importante destacar que este proceso no es lineal sino iterativo: los resultados de cada fase pueden llevar a revisar y enriquecer las fases anteriores. En proyectos ágiles este ciclo se ejecuta en miniatura dentro de cada sprint, mientras que en proyectos tradicionales se ejecuta de manera más extendida al inicio del proyecto:

Figura 2. Proceso de ingeniería de requisitos — fases, actividades e interrelaciones



2.2. Tipos y características de los requisitos

La clasificación más ampliamente aceptada en la ingeniería de software distingue entre requisitos funcionales, no funcionales y de dominio. Esta taxonomía es fundamental para el analista, pues cada tipo requiere técnicas de elicitación y formas de especificación distintas, y tiene implicaciones diferentes para el diseño, la arquitectura y las pruebas del sistema. Ignorar cualquiera de estas categorías durante el levantamiento puede resultar en un sistema que funciona correctamente pero que no satisface las expectativas del cliente en aspectos críticos como el rendimiento, la seguridad o el cumplimiento regulatorio.

Un error muy común en equipos de desarrollo sin experiencia en ingeniería de requisitos es enfocarse exclusivamente en los requisitos funcionales (qué hace el sistema) y descuidar los no funcionales (cómo lo hace). Los atributos de calidad como el rendimiento, la disponibilidad y la seguridad son frecuentemente asumidos implícitamente por el cliente sin verbalizarlos y el analista debe explorarlos activamente

mediante preguntas específicas durante el proceso de elicitación. Descubrirlos tardíamente puede implicar rediseñar la arquitectura completa del sistema, con un costo y un impacto en el cronograma que pueden ser determinantes para el proyecto.

A. Requisitos funcionales

Los requisitos funcionales (RF) describen las funcionalidades específicas que el sistema debe proporcionar: qué debe hacer en respuesta a entradas específicas y cómo debe comportarse en situaciones particulares. En esencia, definen el comportamiento observable del sistema desde la perspectiva del usuario. Se especifican típicamente mediante casos de uso, historias de usuario o escenarios de uso.

Ejemplo de requisito funcional bien redactado:

El sistema debe permitir al usuario registrar una nueva solicitud de servicio ingresando: nombre del cliente (máx. 100 caracteres), descripción del problema (máx. 500 caracteres), nivel de prioridad (alto, medio, bajo) y fecha de creación (automática). Al guardar exitosamente, el sistema debe asignar automáticamente un número de ticket único con formato TK-YYYYMMDD-NNN y mostrar un mensaje de confirmación.

La siguiente tabla presenta ejemplos de cómo se redactan los requisitos funcionales en formato de Historia de Usuario (HU), que es la técnica de especificación más utilizada en metodologías ágiles. Este formato estandarizado facilita la comunicación entre el cliente, el analista y el equipo de desarrollo:

Tabla 3. Ejemplos de historias de usuario para requisitos funcionales

ID	Historia de usuario	Criterio de aceptación	Prioridad
HU-01	Como usuario registrado, quiero iniciar sesión con email y contraseña para acceder al sistema de forma segura.	El sistema valida credenciales en menos de 2 segundos. Bloquea la cuenta tras 5 intentos fallidos. Muestra mensaje de error específico.	Alta
HU-02	Como analista, quiero registrar una solicitud de servicio con todos sus campos para hacer seguimiento al caso del cliente.	El formulario valida campos obligatorios. Asigna número de ticket automático formato TK-YYYYMMDD-NNN. Confirma guardado con notificación.	Alta
HU-03	Como supervisor, quiero ver un reporte de solicitudes pendientes filtradas por prioridad para gestionar la carga de trabajo del equipo.	El reporte se genera en menos de 5 segundos. Permite filtrar por fecha, prioridad y asignado. Exportable en PDF y Excel.	Media
HU-04	Como administrador, quiero gestionar los usuarios del sistema (crear, editar, desactivar) para controlar el acceso a la información.	Solo administradores acceden al módulo. Los cambios quedan registrados en log de auditoría con fecha, hora	Alta

ID	Historia de usuario	Criterio de aceptación	Prioridad
		y usuario que realizó la acción.	
HU-05	Como cliente, quiero recibir una notificación por email cuando mi solicitud cambia de estado para conocer el avance de mi caso.	El email se envía en menos de 1 minuto del cambio de estado. Incluye número de ticket, nuevo estado y enlace de consulta.	Media

B. Requisitos no funcionales

Los requisitos no funcionales (RNF), también llamados atributos de calidad, establecen las restricciones sobre cómo el sistema debe funcionar, no qué debe hacer. Son transversales a todas las funcionalidades e impactan directamente la arquitectura del sistema. Incluyen: rendimiento, seguridad, usabilidad, disponibilidad, escalabilidad, mantenibilidad y portabilidad.

Ejemplo de RNF bien especificado:

El sistema debe responder a cualquier consulta en tiempo máximo de 2 segundos bajo carga normal (hasta 500 usuarios concurrentes). En picos de carga (hasta 1.000 usuarios concurrentes), el tiempo de respuesta no debe superar los 5 segundos. El sistema debe estar disponible el 99,5 % del tiempo (máximo 43,8 horas de inactividad al año).

La especificidad numérica hace que este requisito sea verificable, ya que permite definir criterios de aceptación claros y medibles, facilitando su validación objetiva mediante pruebas y evitando interpretaciones subjetivas.

C. Requisitos de dominio

Los requisitos de dominio son restricciones o condiciones impuestas por el ámbito de aplicación del sistema. Provienen de regulaciones legales, normas industriales o prácticas específicas del negocio.

Ejemplo:

El sistema debe cumplir con la Ley 1581 de 2012 (Protección de Datos Personales de Colombia), implementando mecanismos de consentimiento explícito, derecho al olvido y cifrado de datos sensibles.

Su incumplimiento puede tener consecuencias legales graves, como sanciones regulatorias, responsabilidades contractuales o la imposibilidad de operar dentro del marco normativo establecido.

La siguiente tabla clasifica los diferentes tipos de requisitos con ejemplos concretos del contexto del software empresarial colombiano. Esta clasificación ayuda al analista a asegurarse de que ninguna categoría de requisito quede sin explorar durante el proceso de elicitación:

Tabla 4. Clasificación y ejemplos de tipos de requisitos de software

Tipo	¿Qué describe?	Ejemplo concreto	Fuente típica
Funcional.	Qué hace el sistema.	Registrar usuarios con validación de email único.	Usuarios / cliente.
No funcional – rendimiento.	Qué tan rápido responde.	Tiempo de respuesta < 2 segundos. / 500 usuarios.	Arquitecto / usuario.
No funcional – seguridad.	Cómo protege los datos.	Cifrado AES-256 para datos en reposo.	Normativas / CISO.
No funcional – usabilidad.	Qué tan fácil de usar.	Aprendizaje < 2 horas sin capacitación.	UX / usuarios.
No funcional – disponibilidad.	Cuándo está disponible.	99.5 % uptime anual / SLA definido.	Negocio / SLA.
De dominio.	Restricciones del sector.	Cumplimiento Ley 1581 / GDPR.	Reguladores / legal.
De negocio.	Objetivos organizacionales.	Reducir tiempo de atención en 30 %.	Alta dirección.

Igualmente, y complementando lo anterior, la siguiente tabla profundiza en las diferencias entre los dos tipos más frecuentes de requisitos. Comprender estas diferencias es fundamental para el analista, ya que determinan cómo se elicitan, documentan, verifican y priorizan dentro del proyecto:

Tabla 5. Diferencias entre requisitos funcionales y no funcionales

Criterio	Requisito funcional	Requisito no funcional
Pregunta que responde.	¿Qué hace el sistema?	¿Cómo lo hace?
Perspectiva.	Usuario final.	Arquitecto / QA / operaciones.
Verificación.	Pruebas funcionales / UAT.	Pruebas de rendimiento, seguridad.
Impacto si falla.	Función específica no disponible.	Sistema completo degradado.
Documentación.	Casos de uso / historias de usuario.	Atributos de calidad / SLA.
Visibilidad.	Alta (usuario lo percibe).	Baja (se percibe tardíamente).
Efecto en arquitectura.	Parcial.	Alto y transversal.

3. Elicitación de requisitos de software

La elicitación de requisitos es la actividad más crítica y desafiante dentro del proceso de ingeniería de requisitos. El término “elicitación” (del latín elicere, extraer o sacar a la luz) hace referencia al proceso de descubrir, extraer y recopilar los requisitos del sistema a partir de diversas fuentes: stakeholders humanos, documentos existentes, sistemas actuales y el entorno organizacional. La dificultad fundamental radica en que los usuarios frecuentemente no pueden articular con claridad lo que necesitan, o simplemente no son conscientes de todas sus necesidades hasta que interactúan con prototipos o versiones tempranas del sistema. Este fenómeno, bien documentado en la literatura de ingeniería de software, se conoce como el “problema de los requisitos implícitos”.

El analista experto sabe que los usuarios tienden a describir los síntomas de sus problemas, no las causas raíz. Cuando un usuario dice “el sistema es lento”, el analista debe indagar: ¿lento en qué operaciones específicas?, ¿cuánto tiempo es aceptable?, ¿bajo qué condiciones de carga? Esta capacidad de transformar descripciones vagas en requisitos precisos y verificables es la competencia más diferenciadora del analista de requisitos.

Otros desafíos frecuentes incluyen: diferentes stakeholders con visiones contradictorias sobre los objetivos del sistema (que el analista debe mediar y reconciliar), suposiciones implícitas que los usuarios dan por sentadas sin verbalizarlas (como que “por supuesto el sistema guardará automáticamente cada 5 minutos”), resistencia al cambio por parte de usuarios que temen que el nuevo sistema altere su posición o forma de trabajar y la brecha entre lo que los usuarios dicen que hacen y lo que realmente hacen en su práctica cotidiana. El analista debe anticipar y gestionar

todos estos desafíos de manera proactiva, con una combinación de habilidades técnicas e interpersonales que lo posicionan como un profesional de alto valor en cualquier organización.

3.1. Concepto y análisis de documentos

El análisis de documentos es una técnica de elicitación que consiste en revisar sistemáticamente la documentación existente relacionada con el dominio del problema. Es especialmente valiosa al inicio del proyecto, cuando aún no se tiene acceso a los stakeholders o como complemento a otras técnicas para validar y enriquecer la información recopilada. Permite al analista llegar a las primeras reuniones con los stakeholders con un conocimiento previo del dominio que facilita la comunicación y la formulación de preguntas más pertinentes.

Los documentos que suelen analizarse incluyen:

a) Manuales de procedimientos

Describen paso a paso cómo se ejecutan actualmente los procesos operativos del negocio, permitiendo comprender flujos de trabajo, responsabilidades y reglas implícitas.

b) Formularios en uso

Incluyen formatos físicos o digitales que se emplean para la captura de información, los cuales ayudan a identificar datos requeridos, validaciones y estructuras de entrada.

c) Reportes e informes

Presentan la información que el sistema genera para la toma de decisiones, evidenciando necesidades de control, seguimiento y análisis.

d) Actas y registros históricos

Documentan decisiones, acuerdos, compromisos y cambios definidos en reuniones previas, aportando contexto clave sobre la evolución del proyecto.

e) Contratos y acuerdos

Establecen obligaciones, alcances, niveles de servicio y restricciones legales que deben reflejarse en los requisitos del sistema.

f) Regulaciones del sector

Definen normas y disposiciones legales obligatorias que condicionan el diseño, funcionamiento y operación del sistema.

g) Documentación de sistemas legados

Permite identificar dependencias, integraciones existentes y limitaciones técnicas que deben considerarse en el nuevo desarrollo.

h) Especificaciones de proyectos anteriores

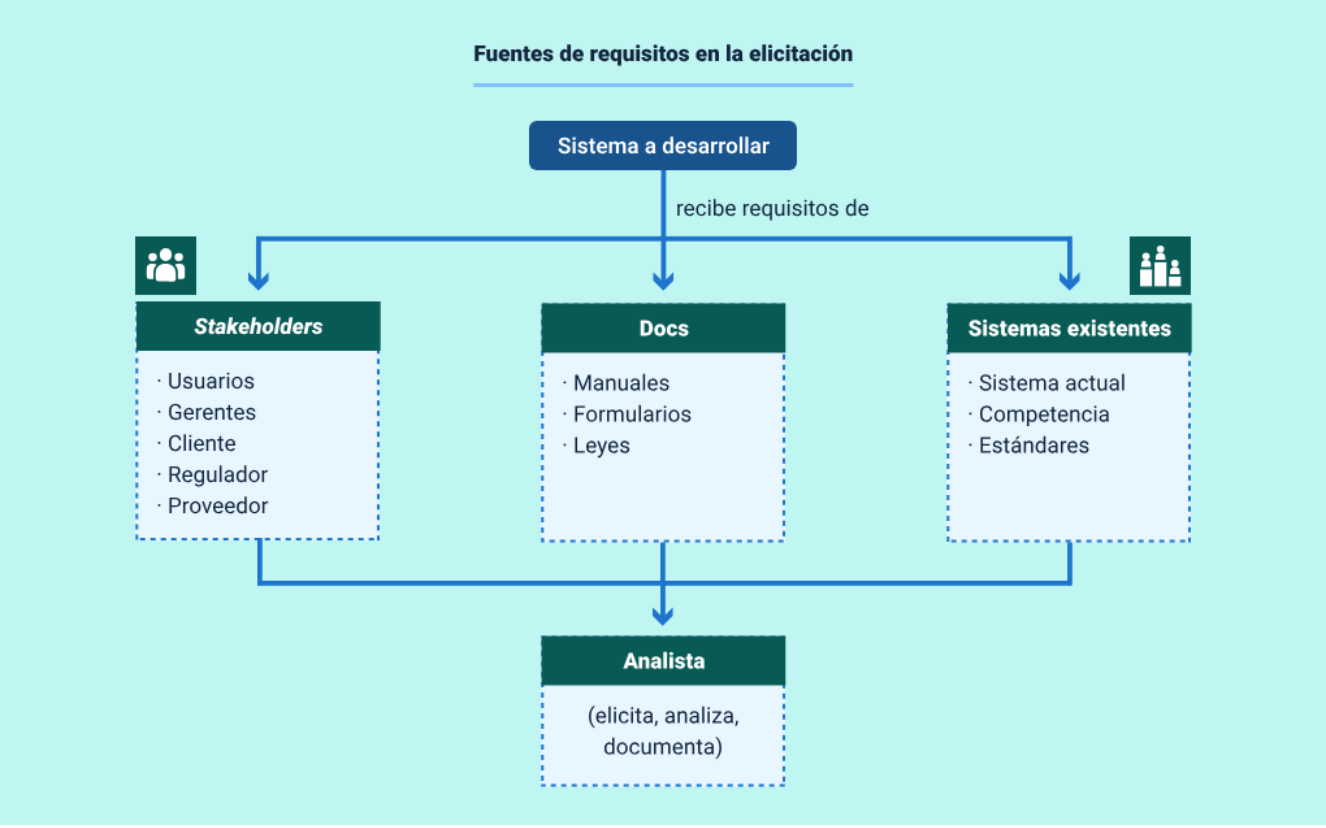
Aportan experiencias previas, lecciones aprendidas y requisitos reutilizables que pueden optimizar el levantamiento actual.

El proceso de análisis de documentos implica: identificar y recopilar todos los documentos relevantes disponibles, revisar su contenido para extraer información sobre el dominio, identificar los procesos actuales y sus flujos de trabajo, detectar

problemas o limitaciones del sistema actual que ya han sido documentados, y validar los hallazgos con los stakeholders para confirmar que la información sigue siendo actual y relevante para el nuevo sistema.

El siguiente esquema presenta un mapa visual de las principales fuentes de requisitos que el analista debe explorar sistemáticamente durante el proceso de elicitación. Cada fuente aporta un tipo diferente de información y requiere técnicas específicas de recolección, lo que explica por qué se recomienda combinar múltiples técnicas en todo proyecto:

Figura 3. Fuentes de requisitos en el proceso de elicitación



4. Técnicas para la recolección de información

La recolección de información constituye el núcleo del proceso de levantamiento de requisitos, ya que de ella depende la calidad, completitud y validez del sistema de software que se construirá. Si los requisitos son el cimiento sobre el cual se edifica la solución, las técnicas de recolección son las herramientas que permiten excavar, comprender y consolidar dicho cimiento. Un sistema técnicamente impecable, pero basado en requisitos incompletos, ambiguos o mal validados, está destinado a generar insatisfacción en el cliente y a fracasar en su propósito de negocio.

Las técnicas de recolección de información son los métodos mediante los cuales el analista obtiene datos, conocimientos y perspectivas de los stakeholders. Ninguna técnica, por sí sola, es capaz de capturar la complejidad total de las necesidades de un sistema de software; cada una actúa como una lente que ilumina ciertos aspectos del dominio mientras deja otros fuera de foco. Por esta razón, la práctica profesional recomienda combinar técnicas complementarias de forma estratégica, en lugar de depender de un único método.

El desafío central del analista de requisitos radica en cerrar la brecha entre lo que el cliente expresa y lo que realmente necesita. En muchos casos, los stakeholders no tienen una visión clara de sus requerimientos hasta que interactúan con prototipos o versiones tempranas del sistema; suelen expresar sus necesidades en términos de soluciones conocidas, omitir información que consideran obvia o confundir deseos con necesidades reales del negocio. La maestría en el uso de las técnicas de recolección es lo que distingue al analista competente de quien se limita a documentar solicitudes sin analizarlas críticamente.

La selección de las técnicas no es arbitraria, sino que debe basarse en el contexto del proyecto. Factores como el perfil de los stakeholders, el tipo de información requerida, las restricciones de tiempo y presupuesto, y el nivel de criticidad del sistema influyen directamente en la estrategia de elicitación. Por ejemplo, en proyectos con cronogramas ajustados, los talleres colaborativos como JAD permiten consensuar requisitos en sesiones intensivas, mientras que en sistemas altamente regulados puede ser necesario aplicar técnicas más exhaustivas y profundas.

Un plan de elicitación bien diseñado integra estas técnicas de manera secuencial y coherente: suele iniciar con el análisis documental y entrevistas exploratorias, continuar con observación directa para validar la comprensión del dominio, avanzar hacia sesiones colaborativas para resolver conflictos y consensuar requisitos, y finalizar con encuestas para validar y priorizar los resultados obtenidos. Este enfoque integral garantiza que ninguna perspectiva relevante quede sin explorar y que los requisitos reflejen fielmente la complejidad real de las necesidades del negocio.

Finalmente, es fundamental que el analista documente no solo los requisitos obtenidos, sino también las técnicas empleadas, los stakeholders involucrados y las decisiones tomadas durante el proceso. Esta trazabilidad metodológica fortalece la credibilidad del proceso de ingeniería de requisitos y permite explicar, justificar y gestionar cambios futuros de manera informada y transparente.

4.1. Técnicas de recolección de información basadas en interacción y observación

Dentro del conjunto de técnicas disponibles, las entrevistas, la observación directa, los cuestionarios y las encuestas ocupan un lugar central por su versatilidad y complementariedad. Mientras las técnicas conversacionales permiten acceder a

percepciones, expectativas y motivaciones, las técnicas observacionales capturan el trabajo real tal como ocurre, y las técnicas estructuradas permiten validar y generalizar la información obtenida a una población más amplia.

La combinación estratégica de estas técnicas proporciona al analista una visión tanto profunda como amplia del dominio del problema: la observación aporta contexto y detalle comportamental; las entrevistas profundizan en las necesidades y razonamientos de los stakeholders; los cuestionarios estructuran y cuantifican la información; y las encuestas permiten validar y priorizar los hallazgos a escala organizacional.

A. Entrevistas

La entrevista es la técnica más utilizada y versátil en el levantamiento de requisitos. Consiste en una conversación estructurada o semi-estructurada entre el analista y uno o más stakeholders, orientada a comprender procesos, necesidades, problemas y expectativas relacionadas con el sistema.

Las entrevistas semi-estructuradas son especialmente recomendadas en proyectos de software, ya que permiten cubrir los temas planificados mientras se exploran aspectos relevantes no anticipados. Pueden realizarse de manera presencial o virtual mediante herramientas como Zoom o Microsoft Teams.

Para una entrevista efectiva se recomienda preparar una guía de preguntas con anticipación, iniciar con preguntas abiertas, aplicar técnicas de escucha activa, grabar la sesión con consentimiento explícito y elaborar un acta de entrevista dentro de las 24 horas siguientes para validarla con el entrevistado.

B. Observación directa

La observación directa consiste en que el analista presencie y registre el trabajo real de los usuarios en su entorno habitual, sin interferir en sus actividades. Esta técnica es especialmente valiosa para capturar el conocimiento tácito: acciones, atajos y prácticas que los usuarios realizan de forma automática y que no suelen mencionar en entrevistas.

Una variante relevante es el protocolo de pensamiento en voz alta (think-aloud), en el cual se le pide al usuario que verbalice sus pensamientos mientras ejecuta sus tareas. Esta combinación permite comprender no solo qué hacen los usuarios, sino también por qué lo hacen de determinada manera.

Dentro de la observación directa se incluye lo siguiente:

- **Observación estructurada**

Utiliza una guía predefinida con categorías, actividades y comportamientos específicos que el analista debe registrar de manera sistemática durante la sesión de observación. Esto garantiza consistencia en la recolección de datos y facilita la comparación entre diferentes sesiones realizadas con distintos usuarios o en distintos contextos.

- **Análisis de tareas**

Complementa la observación al descomponer las actividades del usuario en pasos elementales, permitiendo identificar qué hace el usuario, en qué orden, qué decisiones toma y qué información necesita en cada etapa del proceso. Esta técnica es

fundamental para el diseño de interfaces, la definición de flujos de trabajo y la identificación de puntos críticos del sistema.

- **Estudios contextuales**

Combinan la observación directa con la entrevista en el entorno real de trabajo del usuario. El analista acompaña al usuario durante la ejecución de sus tareas, observando su comportamiento y formulando preguntas en el momento preciso en que se realizan las actividades. Esta técnica proporciona un entendimiento profundo y auténtico del dominio, al eliminar la brecha entre lo que el usuario recuerda y lo que realmente ocurre en la práctica cotidiana.

C. Cuestionarios

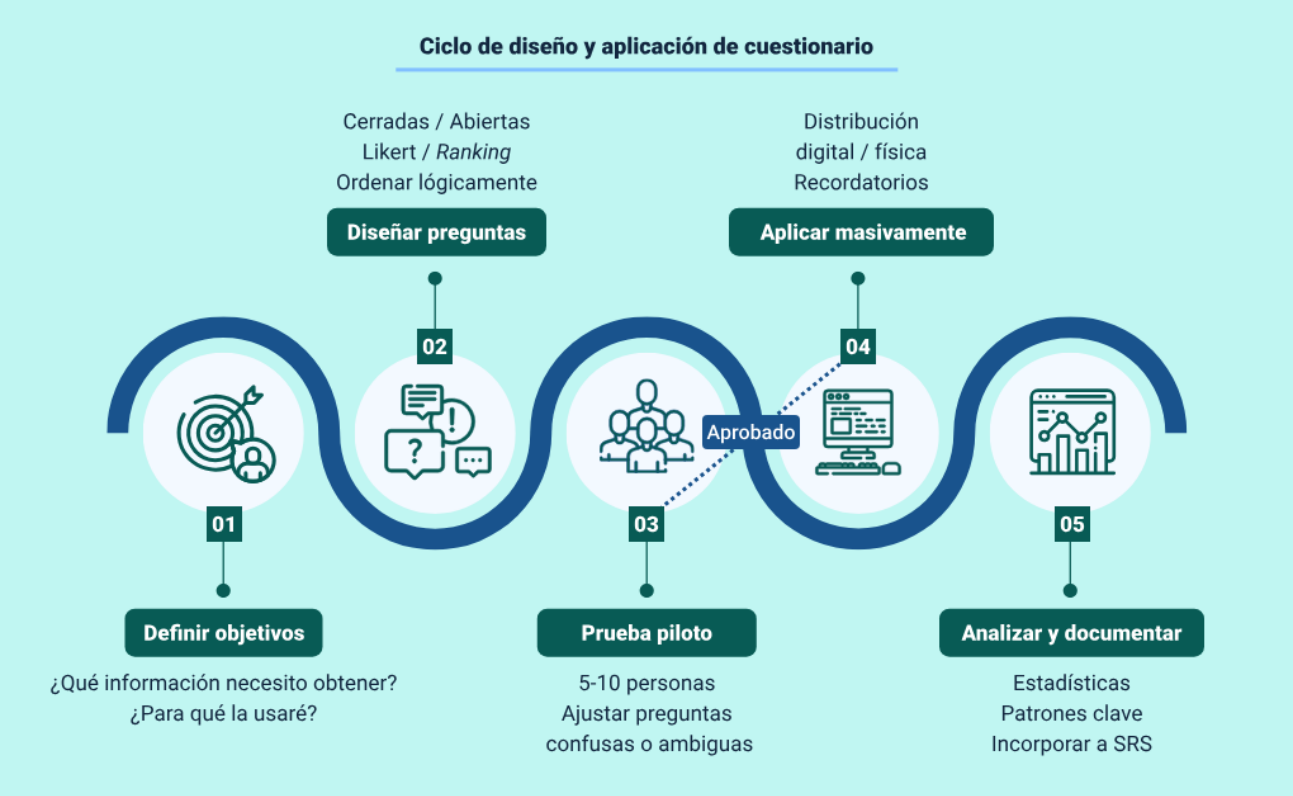
Un cuestionario es un instrumento de recolección de datos compuesto por preguntas diseñadas para obtener información específica de los respondientes. En ingeniería de requisitos se utilizan para recopilar información sobre procesos actuales, frecuencia de actividades, niveles de satisfacción y expectativas frente al nuevo sistema.

Su principal ventaja es la capacidad de llegar a un gran número de personas en poco tiempo y con bajo costo. El diseño de un cuestionario efectivo requiere definir claramente los objetivos, formular preguntas claras y no ambiguas, seleccionar adecuadamente el tipo de preguntas, ordenar los ítems de forma lógica y realizar una prueba piloto antes de su aplicación masiva.

Los tipos de preguntas más utilizados incluyen preguntas cerradas, abiertas, escalas Likert, preguntas de ranking y selección múltiple.

La siguiente imagen esquemática, ilustra el ciclo completo de diseño y aplicación de un cuestionario para el levantamiento de requisitos. Cada etapa del ciclo es indispensable: omitir la prueba piloto, por ejemplo, puede resultar en datos de baja calidad que no sirven para tomar decisiones sobre los requisitos del sistema:

Figura 4. Ciclo de diseño y aplicación de un cuestionario para levantamiento de requisitos



D. Encuestas

La encuesta es el proceso completo de recolección de datos que puede incluir uno o más cuestionarios aplicados a una muestra representativa de la población objetivo. En el levantamiento de requisitos, las encuestas se utilizan para validar

requisitos preliminares, priorizar funcionalidades, medir niveles de satisfacción e identificar necesidades no detectadas mediante otras técnicas.

El uso de herramientas digitales facilita la distribución, el análisis automático de respuestas y la generación de reportes estadísticos. En muchos casos, las encuestas anónimas permiten obtener respuestas más honestas, especialmente cuando se indaga sobre deficiencias del sistema actual.

4.2. Focus group como técnica de recopilación de información

El focus group, también conocido como grupo focal, es una técnica de investigación cualitativa que reúne a un grupo pequeño de participantes (generalmente de 6 a 12 personas con perfil homogéneo) para discutir un tema específico bajo la guía de un moderador especializado. En el levantamiento de requisitos, el focus group permite explorar las opiniones, percepciones, necesidades y expectativas de un grupo representativo de usuarios potenciales del sistema de manera eficiente y económica.

A diferencia de la entrevista individual, el focus group aprovecha la dinámica de grupo: las ideas de un participante estimulan nuevas perspectivas en los demás, generando una comprensión más rica y compleja del problema que la que se obtendría con entrevistas individuales por separado. Sin embargo, también implica riesgos como el pensamiento de grupo (groupthink), donde la presión social puede inhibir que algunos participantes expresen opiniones contrarias a la mayoría, especialmente si hay diferencias jerárquicas entre los participantes.

Conociendo el contexto del focus group, es importante que acceda al siguiente video, donde se explican las etapas que se deben llevar a cabo para una adecuada implementación:

Video 1. Proceso de implementación del focus group



Enlace de reproducción del video

Transcripción del video: Proceso de implementación del focus group
La implementación efectiva de un focus group para el levantamiento de requisitos se apoya en un proceso estructurado compuesto por seis etapas claramente definidas, las cuales permiten obtener información confiable y relevante para el desarrollo del sistema. Cada una cumple una función específica y complementaria, por lo que omitir alguna puede afectar la calidad de los resultados. La primera etapa, la planificación, consiste en definir los objetivos de la sesión, seleccionar participantes con un perfil homogéneo y pertinente, diseñar una guía de

Transcripción del video: Proceso de implementación del focus group

preguntas organizada de lo general a lo específico y preparar el entorno físico o virtual.

La segunda etapa, la apertura, marca el inicio formal de la sesión. En este momento, el moderador presenta los objetivos, establece las reglas de participación y genera un ambiente de confianza que incentive la expresión libre de ideas.

La tercera etapa, la exploración, corresponde al desarrollo del focus group. Aquí se discuten los temas centrales siguiendo la guía, aplicando técnicas de facilitación para profundizar en los puntos clave y equilibrar la participación.

La cuarta etapa, el cierre, tiene como propósito resumir los aportes, validar los consensos alcanzados y agradecer la colaboración de los participantes.

La quinta etapa, el análisis, implica la transcripción de la sesión —si fue grabada con consentimiento— y la realización de un análisis cualitativo para identificar patrones y diferencias.

Finalmente, la sexta etapa, la documentación, consiste en elaborar el informe e incorporar los requisitos identificados al proceso formal de especificación.

En conclusión, aplicar correctamente estas seis etapas convierte al focus group en una herramienta clave para comprender necesidades y fortalecer la toma de decisiones en proyectos de software.

La siguiente tabla compara el focus group con la entrevista individual, las dos técnicas cualitativas más utilizadas en el levantamiento de requisitos. Esta comparación

ayuda al analista a decidir cuál técnica es más apropiada según los objetivos específicos de cada fase del proceso de elicitación:

Tabla 6. Focus group vs. entrevistas: criterios para la selección de la técnica

Criterio	Focus group	Entrevista individual
Participantes por sesión.	6-12 simultáneos.	1-3 por sesión.
Duración típica.	90-120 minutos.	45-90 minutos.
Tipo de información.	Opiniones compartidas, consensos.	Perspectivas individuales profundas.
Dinámica.	Interacción grupal estimula ideas.	Conversación uno a uno.
Riesgo principal.	Pensamiento de grupo (groupthink).	Sesgo del entrevistador.
Moderación requerida.	Facilitador experto.	Analista capacitado suficiente.
Costo por participante.	Bajo (economía de escala).	Alto (más sesiones necesarias).
Recomendado para.	Validar requisitos, priorizar funciones.	Explorar necesidades específicas profundas.

Por su parte, la siguiente tabla presenta las principales herramientas digitales disponibles para cada técnica de recolección de información. Conocer estas herramientas permite al analista planificar el proceso de elicitación con mayor eficiencia, aprovechando las capacidades tecnológicas disponibles en el mercado actual:

Tabla 7. Herramientas digitales para técnicas de recolección de información

Técnica	Herramienta recomendada	Uso principal	Ventaja clave
Encuestas / cuestionarios.	Google Forms, SurveyMonkey.	Distribución masiva y análisis.	Gratuito, fácil de usar.
Entrevistas virtuales.	Zoom, Microsoft Teams.	Sesiones remotas con grabación.	Transcripción automática.
Focus group virtual.	Miro, FigJam, Teams.	Colaboración en tiempo real.	Pizarras colaborativas.
Observación remota.	Hotjar, FullStory.	Análisis de comportamiento web.	Mapas de calor automáticos.
Gestión de requisitos.	JIRA, Azure DevOps.	Documentar y priorizar requisitos.	Trazabilidad integrada.
Prototipado rápido.	Figma, Balsamiq.	Validar interfaces con usuarios.	Colaboración en tiempo real.

4.3. Talleres de Requisitos (JAD - Joint Application Development)

Los talleres JAD (Joint Application Development) son sesiones colaborativas estructuradas donde stakeholders, usuarios, desarrolladores y otros participantes clave trabajan juntos de manera intensiva para definir, analizar y validar requisitos de software. Esta técnica fue desarrollada por IBM en la década de 1970 como una alternativa más eficiente a las entrevistas individuales tradicionales.

A diferencia de las entrevistas uno-a-uno que pueden tomar semanas y la observación directa que es principalmente pasiva, los talleres JAD permiten obtener consenso rápido, resolver conflictos de requisitos en tiempo real y fomentar la colaboración entre diferentes áreas de la organización.

Sus características principales son:

- Sesiones intensivas de 2-5 días consecutivos.
- Participación simultánea de todos los stakeholders clave.
- Facilitador neutral que guía las discusiones.
- Escriba dedicado que documenta decisiones en tiempo real.
- Uso de técnicas visuales y colaborativas (pizarras, post-its, diagramas).
- Toma de decisiones inmediata con autoridad presente.

A. Roles en un taller JAD

El éxito del taller depende de la correcta asignación de roles, por ello se deben tener en cuenta los siguientes, los cuales tienen unas responsabilidades y poseen un perfil ideal:

- **Facilitador**
  - **Responsabilidades**: guiar discusiones, mantener enfoque, resolver conflictos, asegurar participación equitativa.
  - **Perfil ideal**: neutral, experimentado, habilidades interpersonales.
- **Escriba**
  - **Responsabilidades**: documentar decisiones, capturar requisitos en tiempo real.
  - **Perfil ideal**: rápido escribiendo, atención al detalle.

- **Sponsor**
    - **Responsabilidades**: aprobar decisiones, resolver escalamientos, proveer autoridad.
    - **Perfil ideal**: gerente senior con poder de decisión.
- **Usuarios**
    - **Responsabilidades**: expresar necesidades, validar requisitos, proveer casos de uso.
    - **Perfil ideal**: representativos de cada perfil de usuario.
- **Desarrolladores**
    - **Responsabilidades**: evaluar factibilidad técnica, estimar esfuerzo, proponer soluciones.
    - **Perfil ideal**: arquitectos y desarrolladores senior.

Con el fin de ilustrar de manera práctica cómo se desarrollan los talleres JAD y cómo se aplican distintas técnicas colaborativas para la elicitación, análisis y priorización de requisitos, a continuación, se presentan algunos ejemplos representativos que muestran la dinámica de trabajo, la interacción entre los participantes y las herramientas comúnmente utilizadas en este tipo de sesiones:

- **Ejemplo 1. Configuración típica de un taller JAD**

La disposición física del espacio es fundamental para facilitar la colaboración:

Figura 5. Configuración típica de un taller JAD



- **Ejemplo 2. User Story Mapping**

Técnica visual para organizar historias de usuario mostrando el flujo de trabajo y prioridades. Este ejemplo muestra un sistema de e-commerce:

Figura 6. User Story Mapping



- **Ejemplo 3. Diagrama de afinidad**

Método para organizar grandes cantidades de requisitos en grupos relacionados. Los participantes escriben ideas en post-its y colaborativamente las agrupan:

Figura 7. Diagrama de afinidad



- **Ejemplo 4. Matriz de priorización**

Herramienta para evaluar requisitos según criterios de importancia vs. urgencia (matriz de Eisenhower):

Figura 8. Matriz de priorización



Los talleres JAD se complementan con las otras técnicas de elicitación de la siguiente manera:

Tabla 8. Comparativa talleres JAD con otras técnicas de elicitación

Criterio	Entrevistas	Observación	Talleres JAD
Participantes.	1-2 personas.	1-2 observadores.	5-15 stakeholders.
Duración.	30-90 minutos.	2-8 horas.	2-5 días.
Ventaja.	Profundidad individual.	Descubre workflows reales.	Consenso rápido.
Desventaja.	Consume mucho tiempo.	Altera comportamiento.	Difícil coordinar.
Cuándo usar.	Perspectiva de expertos.	Entender procesos actuales.	Validar y resolver conflictos.

Los talleres JAD, igualmente poseen unos aspectos positivos y negativos como son:

Ventajas:

- Reducción dramática del tiempo de elicitación (semanas → días).
- Consenso y resolución de conflictos inmediata.
- Mayor compromiso de los stakeholders con las decisiones tomadas.
- Creatividad potenciada por la interacción grupal.

Desventajas:

- Alto costo de coordinación (difícil reunir a múltiples personas clave).
- Requiere facilitador experto en manejo de grupos.
- Riesgo de dinámicas problemáticas (personalidades dominantes).
- Fatiga de participantes en sesiones muy extensas.

¿Cuándo utilizar talleres JAD?

Los talleres JAD son más efectivos cuando:

- Hay múltiples stakeholders con perspectivas diferentes que deben alinearse.
- Se requiere consenso rápido para avanzar el proyecto.
- Existen conflictos conocidos entre áreas que necesitan resolverse.
- El proyecto es complejo y requiere validación cruzada entre técnicos y usuarios.
- Se busca innovación y creatividad en la definición de requisitos.

Para finalizar, la siguiente tabla sintetiza las principales características de las técnicas de recolección de información más utilizadas en ingeniería de requisitos. Esta comparativa facilita al analista la selección fundamentada de las técnicas más apropiadas según las características específicas del proyecto y el perfil de los stakeholders disponibles:

Tabla 9. Técnicas de recolección de información: comparativa de características

Técnica	Tipo de información	Costo	Tiempo	Participantes	Recomendada cuando...
Entrevistas.	Cualitativa profunda.	Medio.	Alto.	1-3.	Se necesita perspectiva individual detallada.
Observación.	Comportamiento real.	Medio.	Alto.	1-2.	Existe brecha entre lo dicho y lo hecho.
Cuestionarios.	Cuantitativa masiva.	Bajo.	Medio.	Muchos.	Hay muchos usuarios dispersos geográficamente.
Encuestas.	Validación y priorización cuantitativa.	Bajo-medio.	Medio.	Muchos (muestra representativa).	Se requiere validar, priorizar o generalizar requisitos preliminares.
Focus group.	Opiniones grupales.	Medio.	Medio.	6-12.	Se necesita consenso entre grupos de usuarios.

Técnica	Tipo de información	Costo	Tiempo	Participantes	Recomendada cuando...
Talleres JAD.	Colaborativa.	Alto.	Bajo-medio.	10-20.	Se necesitan decisiones rápidas y consenso.

5. Roles en el desarrollo y levantamiento de requisitos

El levantamiento de requisitos es un proceso colaborativo que involucra a múltiples actores con roles, responsabilidades y perspectivas distintas. Comprender quiénes son estos actores, qué papel desempeñan y cómo se relacionan entre sí es fundamental para gestionar eficazmente la comunicación y asegurar que todos los puntos de vista relevantes sean considerados en la definición del sistema.

Una identificación incompleta de los stakeholders puede resultar en requisitos que no reflejan las necesidades de todos los grupos afectados, generando resistencia durante la implementación o un sistema que, aunque técnicamente correcto, no satisface las necesidades reales del negocio. El analista debe ser sistemático y exhaustivo en la identificación de todos los actores desde el inicio del proyecto, utilizando técnicas como el análisis de stakeholders y los mapas de poder-interés.

5.1. Usuarios, actores y stakeholders

En todo proyecto de desarrollo de software coexisten múltiples personas y organizaciones con intereses, expectativas y niveles de influencia distintos sobre el sistema que se va a construir. Comprender con precisión quiénes son, qué rol desempeñan y cómo se diferencian entre sí es una de las primeras responsabilidades del analista de requisitos, ya que de esta comprensión depende la calidad y la representatividad de la información que se recopile durante el proceso de elicitación. Los conceptos de usuario, actor y stakeholder son frecuentemente confundidos o utilizados como sinónimos en el lenguaje cotidiano de los equipos de desarrollo, pero en la ingeniería de software tienen significados precisos y complementarios que el

profesional debe dominar. Ignorar a alguno de estos actores durante el levantamiento de requisitos es una de las causas más frecuentes de que el producto final no satisfaga a todos los grupos involucrados, generando resistencia en la implementación y costosas revisiones posteriores. Por ello, identificar, clasificar y gestionar adecuadamente a cada uno de estos actores desde el inicio del proyecto es una práctica que el analista debe ejecutar con rigor y sistematicidad.

Por lo anterior, se explica cada uno:

A. Usuario

El usuario es la persona que interactúa directamente con el sistema de software para realizar sus tareas cotidianas. Es quien introduce datos, consulta información, genera reportes y toma decisiones apoyado en el sistema. Existen diferentes tipos de usuarios según su nivel de experiencia técnica (novatos, intermedios, expertos), su frecuencia de uso (ocasionales y habituales) y sus objetivos específicos dentro del sistema.

Los usuarios son la fuente más directa de información sobre los requisitos funcionales, pues son quienes mejor conocen los procesos que el sistema debe soportar. Sin embargo, frecuentemente tienen dificultades para articular sus necesidades en términos abstractos; prefieren describirlas mediante ejemplos concretos y casos de uso específicos, por lo que el analista debe facilitar su expresión mediante técnicas adecuadas como el prototipado, los escenarios de uso y el role-playing.

Ejemplo:

El usuario percibe confianza en la marca al saber que sus datos son tratados de manera ética, utilizados únicamente para los fines informados y gestionados con transparencia, lo que le brinda seguridad y control sobre su información.

B. Actor (en el contexto UML)

En el contexto de los casos de uso (técnica de especificación UML), un actor representa un rol que una persona, sistema externo u organización desempeña al interactuar con el sistema en desarrollo. A diferencia del usuario (persona específica), el actor es una abstracción: la misma persona puede desempeñar múltiples actores en el sistema, y el mismo actor puede ser desempeñado por múltiples personas con características similares.

Ejemplo:

En un sistema de gestión hospitalaria, la persona “María García” puede ser tanto “Médico tratante” (cuando registra diagnósticos y ordena exámenes) como “Usuario del sistema de citas” (cuando agenda sus propias citas de formación).

Los actores se representan en los diagramas de casos de uso como figuras humanas etiquetadas con el nombre del rol, lo que facilita la comprensión visual de las interacciones del sistema.

C. Stakeholder

El término stakeholder (parte interesada) hace referencia a cualquier persona, grupo u organización que tiene un interés en el sistema, ya sea porque se beneficia de él, lo financia, lo desarrolla, lo regula o se ve afectado por su implementación. Los

stakeholders no necesariamente interactúan directamente con el sistema; su interés puede ser de carácter económico, legal, operativo o estratégico.

Ejemplos de stakeholders en un proyecto de software empresarial:

La dirección general (define objetivos estratégicos y aprueba el presupuesto), el departamento legal (impone restricciones de cumplimiento normativo), los usuarios finales (utilizarán el sistema diariamente), los administradores de sistemas (responsables del despliegue y mantenimiento), los reguladores externos (definen restricciones legales y normativas como la Ley 1581 de datos personales en Colombia) y los proveedores cuya integración puede ser necesaria.

Para complementar los anteriores conceptos, la siguiente tabla aclara las diferencias conceptuales entre los tres términos que con mayor frecuencia generan confusión en los equipos de proyecto: usuario, actor y stakeholder. Dominar esta distinción es esencial para comunicarse con precisión en el ámbito profesional del desarrollo de software:

Tabla 10. Diferencias entre usuario, actor y stakeholder

Concepto	Definición	Ejemplo concreto	Relación con el sistema	Fuente de requisitos
Usuario.	Persona que usa el sistema directamente.	Cajero en un banco.	Directa y frecuente.	Requisitos funcionales.

Concepto	Definición	Ejemplo concreto	Relación con el sistema	Fuente de requisitos
Actor (UML).	Rol modelado en casos de uso.	Rol: cajero, supervisor, auditor.	Directa (en casos de uso).	Flujos de interacción.
Stakeholder.	Cualquier parte con interés en el sistema.	Director financiero, regulador SFC.	Directa o indirecta.	Requisito de negocio / dominio.

5.2. Cliente líder, dueño del producto, equipo de desarrollo y analista

En los proyectos de desarrollo de software, la correcta definición y articulación de los roles involucrados en la gestión y el levantamiento de requisitos es un factor crítico para el éxito del producto final. La diversidad de intereses, conocimientos y responsabilidades hace necesario establecer claramente quién representa al negocio, quién toma decisiones sobre el alcance, quién construye la solución y quién garantiza que las necesidades del cliente se traduzcan adecuadamente en especificaciones técnicas.

En este contexto, roles como el cliente líder, el dueño del producto, el equipo de desarrollo y el analista de requisitos cumplen funciones complementarias que permiten asegurar una comunicación fluida entre los usuarios, el negocio y el equipo técnico. Cada uno aporta una perspectiva distinta al proceso de elicitación, validación y priorización de requisitos, contribuyendo a reducir ambigüedades, gestionar expectativas y alinear el producto con los objetivos estratégicos de la organización.

Comprender las responsabilidades, autoridad y competencias de cada uno de estos roles resulta fundamental para evitar solapamientos, conflictos de decisión o vacíos de información que puedan afectar la calidad del producto. A continuación, se describen de manera detallada las características y funciones de cada uno de estos actores dentro del proceso de desarrollo:

A. Cliente líder

El cliente líder (también conocido como representante de usuarios o power user) es un usuario con alto conocimiento del dominio del negocio que actúa como enlace entre el grupo de usuarios y el equipo de desarrollo. Este rol es fundamental en proyectos donde el número de usuarios es grande y no es práctico involucrar a todos directamente en el proceso de elicitación. Su selección debe ser cuidadosa, ya que representa los intereses de sus colegas y sus decisiones vinculan al grupo de usuarios.

El cliente líder tiene autoridad para tomar decisiones sobre los requisitos en nombre de su grupo de usuarios. Sus responsabilidades incluyen:

- **Autoridad en la toma de decisiones**

Tiene la facultad de decidir sobre los requisitos en representación de su grupo de usuarios, vinculando sus decisiones al conjunto del colectivo.

- **Participación en el levantamiento y validación**

Interviene activamente en las sesiones de elicitación y validación para asegurar que los requisitos reflejen las necesidades reales del negocio.

- **Revisión y aprobación de requisitos**

Revisa y aprueba la especificación de requisitos, garantizando su coherencia y alineación con los procesos del usuario.

- **Priorización de funcionalidades**

Define el orden de las funcionalidades desde la perspectiva del valor y la utilidad para los usuarios finales.

- **Facilitación del acceso a usuarios**

Actúa como puente entre el equipo de desarrollo y otros usuarios cuando se requiere información adicional o validaciones específicas.

- **Punto de contacto del negocio**

Es la referencia principal para resolver dudas relacionadas con los procesos del negocio durante el desarrollo del proyecto.

Para desempeñar este rol efectivamente, el cliente líder debe reunir varias condiciones: conocimiento profundo y actualizado de los procesos del negocio, capacidad para comunicarse tanto con usuarios no técnicos como con el equipo de desarrollo, disponibilidad real de tiempo para participar en el proyecto (lo que implica un acuerdo formal con su jefatura), autoridad reconocida por sus colegas usuarios, y habilidades para gestionar conflictos y llegar a consensos cuando existen intereses divergentes entre grupos de usuarios.

B. Product Owner (dueño del producto)

El Product Owner (PO) es un rol propio de los frameworks ágiles, especialmente de SCRUM. Es el responsable de maximizar el valor del producto y de gestionar el product backlog (lista priorizada de requisitos). Actúa como el representante del negocio dentro del equipo de desarrollo, tomando decisiones sobre qué funcionalidades se construirán, en qué orden y con qué criterios de aceptación. Es el único que puede agregar, modificar o repriorizar ítems del product backlog.

El PO tiene tres responsabilidades fundamentales que el Scrum Guide define con precisión:

- **Definición de ítems del product backlog**

Establece claramente los ítems del product backlog mediante user stories bien formuladas y criterios de aceptación medibles y verificables.

- **Priorización basada en valor y riesgo**

Ordena los ítems considerando el valor para el negocio, el riesgo técnico y las dependencias entre funcionalidades, asegurando un uso óptimo de los recursos.

- **Aseguramiento de la comprensión del equipo**

Verifica que el equipo de desarrollo comprenda los requisitos con el nivel de detalle necesario para su correcta implementación en cada sprint.

A diferencia del cliente líder, el PO tiene plena autoridad para tomar decisiones sobre el producto sin consultar a la dirección en cada caso. Esta autoridad es indispensable para que el equipo ágil pueda planificar y ejecutar sprints con certeza,

sin interrupciones causadas por la falta de una persona con poder de decisión sobre el alcance.

C. Equipo de desarrollo

El equipo de desarrollo está compuesto por los profesionales responsables de construir el sistema: programadores, diseñadores UX/UI, arquitectos de software e ingenieros de pruebas (QA). En los procesos ágiles, el equipo es multifuncional y autoorganizado: sus miembros poseen colectivamente todas las habilidades necesarias para construir el producto y deciden internamente cómo organizar su trabajo para alcanzar los objetivos de cada sprint sin supervisión directa de un gerente de proyecto.

El rol del equipo de desarrollo en el levantamiento de requisitos es activo, no pasivo. Los desarrolladores participan en las sesiones de refinamiento del backlog, hacen preguntas que clarifican los requisitos antes de comenzar la implementación, identifican dependencias técnicas y ambigüedades que el analista puede no haber detectado, estiman el esfuerzo de implementación de cada historia de usuario, y proponen soluciones técnicas alternativas cuando los requisitos originales presentan complejidad excesiva o riesgos técnicos significativos que el cliente no había anticipado.

D. Analista de requisitos

El analista de requisitos es el profesional especializado en el proceso de ingeniería de requisitos. Su función principal es actuar como puente entre el mundo del negocio y el mundo técnico, traduciendo las necesidades del cliente en especificaciones comprensibles para el equipo de desarrollo. Es el responsable de garantizar que los requisitos sean completos, consistentes y verificables, y de gestionar los cambios que inevitablemente surgen durante el desarrollo del proyecto.

Las competencias clave del analista abarcan tanto habilidades técnicas (dominio de técnicas de elicitación, modelos de especificación UML, herramientas de gestión de requisitos como JIRA o Azure DevOps, estándares IEEE 830 y 29148) como interpersonales (comunicación efectiva con personas de diferentes perfiles, escucha activa, facilitación de reuniones, negociación de prioridades y gestión constructiva de conflictos entre stakeholders con intereses divergentes).

El analista también es responsable de la trazabilidad de requisitos: asegurar que cada requisito pueda vincularse a su origen (stakeholder o documento de negocio) y a los artefactos de diseño, código y pruebas que lo implementan. Esta trazabilidad es fundamental para evaluar el impacto de los cambios, gestionar el alcance del proyecto y demostrar el cumplimiento de regulaciones en sectores altamente controlados como el financiero, el de salud o el gubernamental.

La siguiente tabla resume las responsabilidades, el nivel de autoridad y la habilidad más crítica de cada rol. Esta información es la base para construir la matriz RACI del proyecto —Responsable, Aprobador, Consultado, Informado— que define con precisión quién participa en cada actividad del levantamiento de requisitos y en qué capacidad:

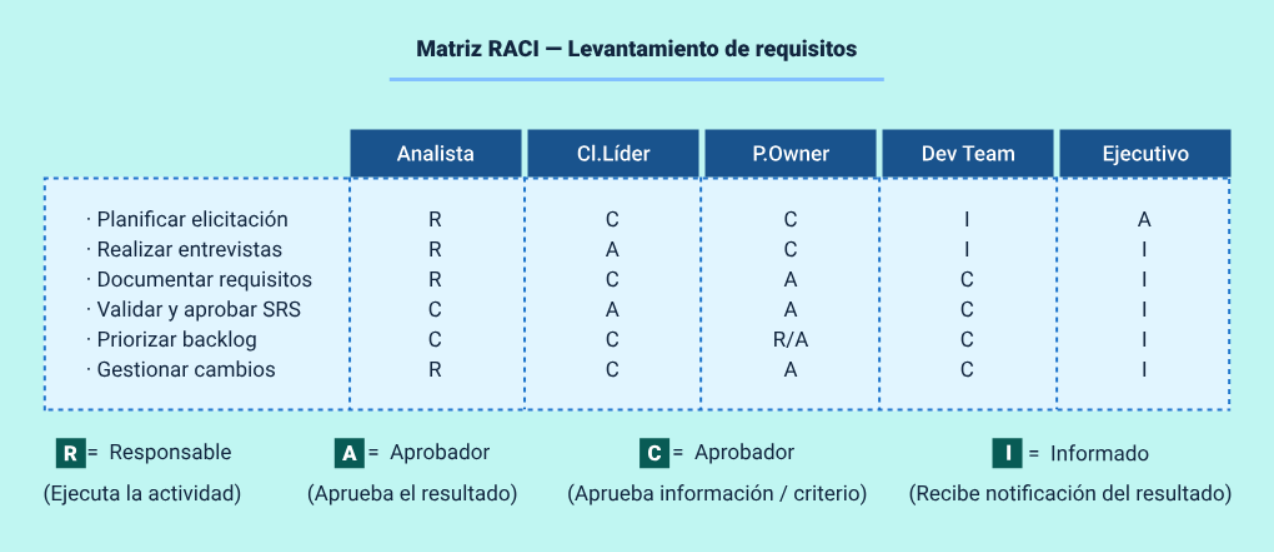
Tabla 11. Roles en el levantamiento de requisitos: responsabilidades y autoridad

Rol	Contexto	Responsabilidad principal	Autoridad sobre requisitos	Habilidad clave
Cliente líder.	Proyectos tradicionales y ágiles.	Representar usuarios, validar requisitos.	Alta (en nombre de usuarios).	Conocimiento del negocio.

Rol	Contexto	Responsabilidad principal	Autoridad sobre requisitos	Habilidad clave
Product Owner.	Proyectos ágiles (SCRUM).	Gestionar y priorizar product backlog.	Total sobre el producto.	Visión de producto y negocio.
Equipo desarrollo.	Todos los contextos.	Implementar requisitos, estimar esfuerzo.	Ninguna (receptor).	Habilidades técnicas.
Analista requisitos.	Todos los contextos.	Elicitar, documentar y gestionar.	Facilita y coordina.	Comunicación y análisis.
Stakeholder ejecutivo.	Proyectos estratégicos.	Definir objetivos, aprobar presupuesto.	Alta (objetivos estratégicos).	Visión estratégica.

De igual manera, la siguiente imagen ilustra la matriz RACI aplicada a las cinco actividades principales del proceso de levantamiento de requisitos, mostrando cómo se distribuyen las responsabilidades entre los roles. Esta herramienta es indispensable para proyectos con múltiples stakeholders y evita ambigüedades sobre quién toma las decisiones en cada etapa:

Figura 9. Matriz RACI para las actividades de levantamiento de requisitos



Matriz RACI — Levantamiento de requisitos

	Analista	Cl.Líder	P.Owner	Dev Team	Ejecutivo
· Planificar elicitación	R	C	C	I	A
· Realizar entrevistas	R	A	C	I	I
· Documentar requisitos	R	C	A	C	I
· Validar y aprobar SRS	C	A	A	C	I
· Priorizar backlog	C	C	R/A	C	I
· Gestionar cambios	R	C	A	C	I

R = Responsable (Ejecuta la actividad)

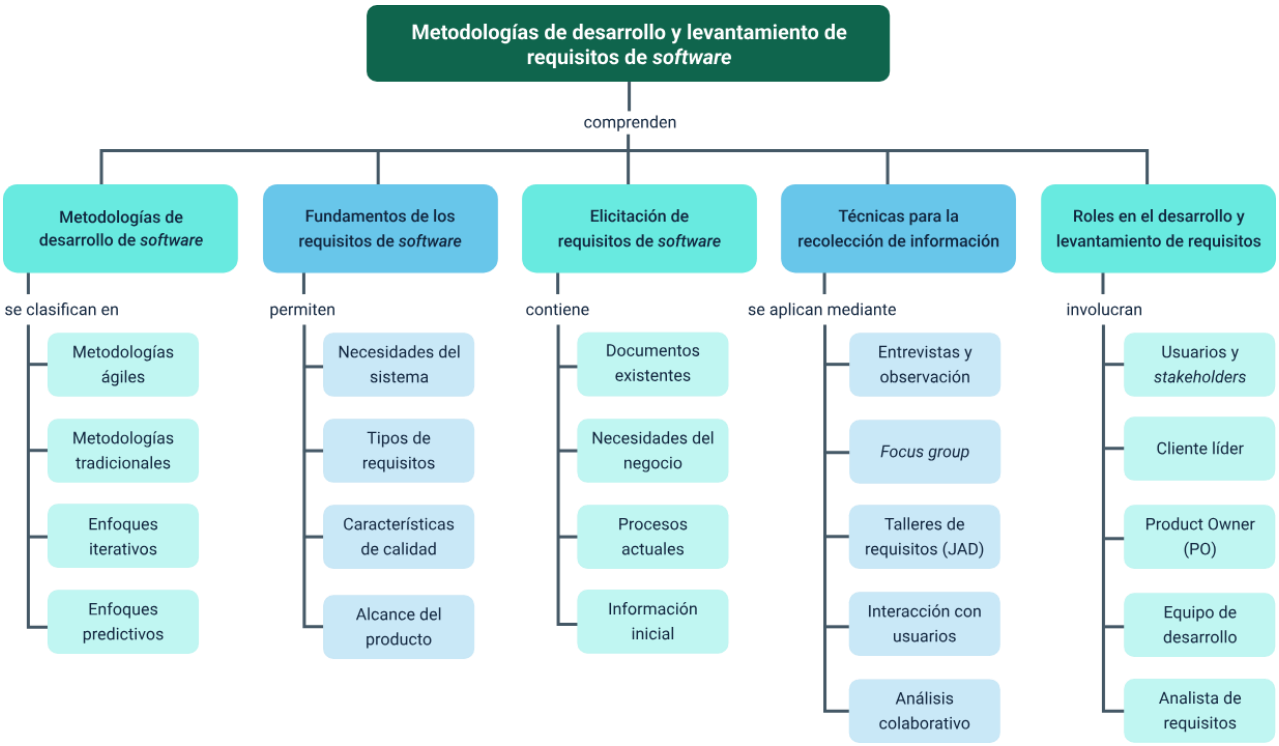
A = Aprobador (Aprueba el resultado)

C = Aprobador (Aprueba información / criterio)

I = Informado (Recibe notificación del resultado)

Síntesis

La formación en metodologías de desarrollo de software y levantamiento de requisitos permite construir una comprensión integral del proceso de creación de sistemas informáticos, articulando fundamentos teóricos y aplicación práctica. A lo largo del componente se analizan metodologías tradicionales y ágiles, sus características, ventajas y limitaciones, entendidas como medios estratégicos para responder a distintos contextos de proyecto. Asimismo, se profundiza en los fundamentos de los requisitos de software, sus tipos y su importancia crítica en el éxito del desarrollo, junto con el dominio de técnicas de elicitación y recolección de información como entrevistas, observación, focus group y talleres de requisitos. Finalmente, se abordan los roles involucrados —usuarios, actores, stakeholders, cliente líder, Product Owner, equipo de desarrollo y analista— destacando la relevancia de la comunicación, la coordinación y las habilidades humanas, así como la necesidad de un aprendizaje continuo para enfrentar la evolución constante de la ingeniería de software y mantener al usuario como eje central de las decisiones técnicas.


Metodologías de desarrollo y levantamiento de requisitos de *software*
comprenden
Metodologías de desarrollo de *software*
se clasifican en
Metodologías ágiles
Metodologías tradicionales
Enfoques iterativos
Enfoques predictivos
Fundamentos de los requisitos de *software*
permiten
Necesidades del sistema
Tipos de requisitos
Características de calidad
Alcance del producto
Elicitación de requisitos de *software*
contiene
Documentos existentes
Necesidades del negocio
Procesos actuales
Información inicial
Técnicas para la recolección de información
se aplican mediante
Entrevistas y observación
Focus group
Talleres de requisitos (JAD)
Interacción con usuarios
Análisis colaborativo
Roles en el desarrollo y levantamiento de requisitos
involucran
Usuarios y *stakeholders*
Cliente líder
Product Owner (PO)
Equipo de desarrollo
Analista de requisitos

Glosario

Actor: rol que desempeña una persona, sistema externo u organización al interactuar con el sistema en desarrollo. Se representa en diagramas de casos de uso de UML como una figura humana con el nombre del rol.

Backlog: lista priorizada y dinámica de todos los requisitos, funcionalidades y mejoras pendientes de un producto de software. En SCRUM es gestionada exclusivamente por el Product Owner.

Elicitación: proceso sistemático de descubrir, extraer y recopilar los requisitos de un sistema a partir de múltiples fuentes (stakeholders, documentos, sistemas existentes), utilizando diversas técnicas de recolección de información.

Requisito funcional: descripción de una funcionalidad específica que el sistema debe proporcionar, definiendo qué debe hacer el sistema en respuesta a entradas o situaciones particulares, desde la perspectiva observable del usuario.

Requisito no funcional: atributo de calidad o restricción que define cómo debe funcionar el sistema (rendimiento, seguridad, usabilidad, disponibilidad, escalabilidad), en contraste con qué debe hacer. Tiene alto impacto en la arquitectura del sistema.

Sprint: iteración de duración fija (1 a 4 semanas) en SCRUM, al final de la cual el equipo debe entregar un incremento de software potencialmente desplegable que agrega valor al producto y puede ser mostrado al cliente para obtener retroalimentación.

Trazabilidad: capacidad de vincular cada requisito con su origen (stakeholder, documento de negocio o regulación) y con los artefactos de diseño, código y pruebas que lo implementan. Facilita la gestión de cambios y el cumplimiento regulatorio.

Referencias bibliográficas

Boehm, B. & Turner, R. (2004). Balancing agility and discipline: a guide for the perplexed. Addison-Wesley Professional.

Cohn, M. (2004). User stories applied: for agile software development. Addison-Wesley Professional.

IEEE Computer Society. (2014). IEEE guide to the software engineering body of knowledge (SWEBOK V3.0). IEEE Press.

Leffingwell, D. (2021). SAFe 5.0 distilled: achieving business agility with the scaled agile framework. Addison-Wesley Professional.

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: un enfoque práctico (8.a ed.). McGraw-Hill Education.

Robertson, S., & Robertson, J. (2012). Mastering the requirements process: getting requirements right (3rd ed.). Addison-Wesley Professional.

Rubin, K. S. (2012). Essential Scrum: a practical guide to the most popular agile process. Addison-Wesley Professional.

Sommerville, I. (2019). Engineering software products: an introduction to modern software engineering. Pearson Education.

Standish Group. (2015). CHAOS Report. Standish Group International.

Wiegers, K., & Beatty, J. (2013). Software requirements (3rd ed.). Microsoft Press.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema de Recursos Educativos Digitales (RED)	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Veimar Celis Melendez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima

Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima